

DDA5001 Machine Learning

Large Language Models (Part I)

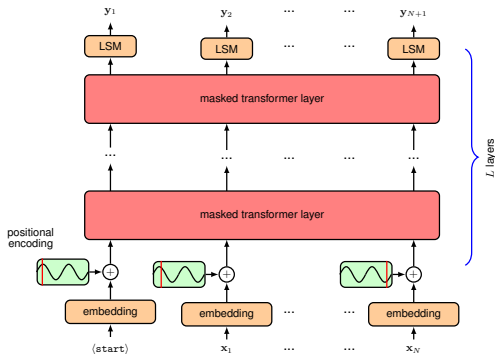
Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Large Language Models (LLMs)

- Contemporary LLMs are usually built on (decoder) transformer architecture.



Agenda

Autoregressive Modeling

Pre-training

Post-training

Conditional Probability is All We Need

- ▶ Recall that classification can be transferred to finding conditional probability. This is the foundation of logistic regression.
- ▶ Language models are trained to **predict the next token**, which can also be modeled as maximizing conditional probability.

Suppose that our input **prompt** (i.e., input tokens) is $\mathbf{x}$ and the language model will generate the response $\mathbf{y} \in \mathbb{R}^m$ with probability

$$\mathbb{P}[\mathbf{y}|\mathbf{x}].$$

In LLMs, it models the conditional probability using a transformer with autoregressive probability decomposition:

$$\mathbb{P}_{\theta}[\mathbf{y}|\mathbf{x}].$$

- ▶ $\mathbb{P}_{\theta}$ is the **transformer model**, which predicts the conditional probability using the logistic regression in the final layer.
- ▶ θ contains transformer's trainable parameters.

Autoregressive Modeling of Conditional Probability

In LLM, it models the conditional probability using a transformer with autoregressive probability decomposition:

$$\mathbb{P}_{\theta}[\mathbf{y}|\mathbf{x}] = \prod_{j=1}^m \mathbb{P}_{\theta}[y_j|\mathbf{x}, \mathbf{y}_{1:j-1}].$$

y_j is the j -th token in the output $\mathbf{y}$.

- ▶ The above decomposition is always true. It is the foundation of LLMs.
- ▶ Generating the whole response $\mathbf{y}$ can be decomposed into **autoregressively predicting the next token**.

Training: Self-Labeling / Next-Token Prediction

Each distribution p_i is for sampling the next token.

During training: The input is $\langle \text{start} \rangle$ plus $\{x_1, \dots, x_n\}$ ($n + 1$ tokens). The output is $\{p_1, \dots, p_n, p_{n+1}\}$, i.e., $n + 1$ distributions.

- ▶ Distribution p_1 should predict token x_1 .
- ▶ ...
- ▶ Distribution p_n should predict token x_n .
- ▶ Distribution p_{n+1} should predict the **end of sequence** $\langle \text{EOS} \rangle$ special token.

The intuition is that for any input sequence $x_{\leq i}$, the next token x_{i+1} acts as the groundtruth label. Namely, the generated distribution p_{i+1} using input $x_{\leq i}$ should predict the true label/token x_{i+1} .

- ▶ The true label x_{i+1} is represented as a **one-hot** encoding $(0, 0, \dots, 1, \dots, 0) \in \mathbb{R}^V$ with 1 at the token position x_{i+1} in the vocabulary.
- ▶ During training, we encourage the generated p_{i+1} to be **as close as possible** to the above one-hot label of x_{i+1} .
- ▶ This is called **self-labeling**, or **next-token prediction**.

Training Problem Formulation

- ▶ The learning problem of LLMs is just a **maximum likelihood estimation** (MLE).

That is, given a set of training data pairs $\mathcal{D} = \{\mathbf{x}_i\}$, the MLE learning problem of LLMs can be formulated as minimizing the negative log-likelihood of the generation probability:

$$\hat{\boldsymbol{\theta}} \leftarrow \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \mathcal{L}(\boldsymbol{\theta}) = \sum_{i \in \mathcal{D}} \sum_{j=1}^m -\log(\mathbb{P}_{\boldsymbol{\theta}}[x_{i,j} | \mathbf{x}_{i,1:j-1}]) . \quad (\text{P})$$

- ▶ This training formulation of autoregressive model is called **self-supervised** learning, using the later tokens in a sentence as supervision.

Inference

- ▶ After an LLM is trained, the text generation process is called **inference**.
- ▶ Inference is the forward passes of an LLM, i.e., generating the response y given input prompt x .

Inference process of LLMs: We will only use the last classification to predict next token:

$$\text{softmax}(z_{end}^T W^L),$$

where z_{end} is the learned feature of the last token of the input.

- ▶ We need to input **all** the tokens of the prompt even though we only use the last feature for classification, due to attention mechanism.
- ▶ The predicted token is **appended** to the current sequence. This updated sequence (the prompt + already-generated tokens) is fed back into the Transformer to predict the subsequent token.
- ▶ The generation is finished when we reach the **preset** maximum number of generation length, or when it automatically generates the special **end of sequence** <EOS> token.

Sampling Strategies in Inference

- ▶ The generated probability distribution $\text{softmax}(\mathbf{z}_{end}^\top \mathbf{W}^L)$ fully determines the next token to choose from the vocabulary.

Sampling strategies:

- ▶ **Greedy Decoding**: Selects the token with the highest **local** probability at each step: $y_t = \operatorname{argmax} P(y|y_{<t}, x)$.
 - ▶ Note: This does **not** guarantee finding the sequence with the maximum **global** probability $\max \mathbb{P}_\theta[\mathbf{y}|\mathbf{x}]$.
- ▶ **Beam Search**: Maintains the top B most promising candidate sequences at each time/decoding step.
 - ▶ This explores a larger search space than greedy decoding and generally yields higher-quality sequences.

Agenda

Autoregressive Modeling

Pre-training

Post-training

Pre-training Overview

Overview:

- ▶ Pre-training is to train an LLM **from scratch** by optimizing the learning objective (P) on a **large corpus** (i.e., training dataset).
- ▶ This looks like a standard training procedure. However, it contains so many non-trivial engineering steps, huge computing resources, and manpower resources due to **truly large-scale** feature.

We quickly go through: data, model architecture, scaling law, training setting for pre-training.

We will take **Llama 3** developed by Meta as example.

Pre-training Dataset

Llama 3's pre-training uses **15T** carefully crafted data (15 trillion tokens) from the **internet**.

The processing of data includes:

- ▶ Safety filtering.
- ▶ Text extraction and cleaning: extract high quality text from HTML.
- ▶ De-duplication.
- ▶ Quality filtering.
- ▶ Code and math reasoning data.
- ▶ Multilingual data: 176 languages.

Data mix:

- ▶ 50% general knowledge tokens.
- ▶ 25% mathematical and reasoning tokens.
- ▶ 17% code tokens.
- ▶ 8% multilingual tokens.

Model Architecture

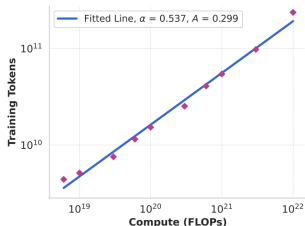
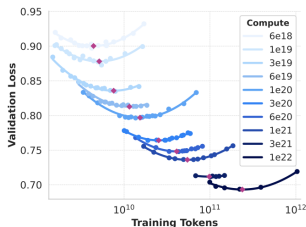
- ▶ Llama 3 uses a standard, dense (decoder) **Transformer** architecture.
- ▶ Use **group query attention** with 8 key-value heads to accelerate inference speed.
- ▶ Vocabulary is of size **128K**, i.e., K in the final layer logistic regression classifier has dimension 128K.
- ▶ Llama 3 has 8B, 70B, 405B models (trainable parameters of the transformer).

Overview of the key hyperparameters of Llama 3:

	8B	70B	405B
Layers	32	80	126
Model Dimension	4,096	8192	16,384
FFN Dimension	6,144	12,288	20,480
Attention Heads	32	64	128
Key/Value Heads	8	8	8
Peak Learning Rate	3×10^{-4}	1.5×10^{-4}	8×10^{-5}
Activation Function	SwiGLU		
Vocabulary Size	128,000		
Positional Embeddings	RoPE ($\theta = 500,000$)		

Scaling Law

- Scaling law is to predict the training loss of (P) based on training tokens and model size trade-off, given a fixed compute budget (measured by FLOPS).



- The left figure uses quadratic function to do fitting.
- The right figure takes the red compute-optimal points in the left figure to fit a power law relationship:

$$N^*(C) = A \cdot C^\alpha.$$

Here, C is compute FLOPS (x-axis) and N^* is the optimal training tokens (y-axis). Determining N^* also implicitly determines the optimal model size.

Training Recipe

Llama 3-405B is pre-trained using

- ▶ AdamW optimizer.
- ▶ 8×10^{-5} initial lr, and use cosine decay to 8×10^{-7} .
- ▶ It uses an **increasing batch size** to stabilize training process. The batch size for computing the stochastic gradient is starting from 4M, to 8M, and finally to 16M.
- ▶ It also increases the sequence length, starting from 4096 to final 128K context window, in order to learn long context understanding.

Agenda

Autoregressive Modeling

Pre-training

Post-training

SFT: Supervised Finetuning

- ▶ A pre-trained LLM can be regarded as having general knowledge.
- ▶ Sometimes, we need to adapt an LLM to a specific domain.
- ▶ Additionally, pre-trained model is **not** capable of understanding human instructions. We need to adapt it to follow human instructions.

↪ The need of **supervised finetuning (SFT)**, or sometimes simply called finetuning.

- ▶ SFT has exactly the same learning problem formulation (P). The only difference is that it is done on a **much smaller** finetuning dataset, compared to the pre-training dataset.
- ▶ Our final project part II is an SFT task. The Math500 dataset is commonly used for improving the mathematical capability of an LLM.

Memory Analysis for Finetuning LLMs

To finetune an LLM with M billion parameters using AdamW optimizer and mixed precision training approach, one needs to store:

- ▶ Float16 model: $2M$ GB
- ▶ Float32 model: $4M$ GB
- ▶ Float32 gradient: $4M$ GB
- ▶ Float32 momentum: $4M$ GB
- ▶ Float32 second moment: $4M$ GB
- ▶ Activation values.

In total, one needs at least $18M$ GB GPU RAM, plus additional memory for storing activations.

Examples:

- ▶ For Llama 3-8B, one needs at least 144GB GPU RAM, necessitating $2 \times A100-80GB$.
- ▶ For Llama 3-70B, $8 \times A100-80GB$ is even not enough...
- ▶ This is why we can only tune a very small model.

PO: Preference Optimization of LLMs

- **Preference optimization (PO)** is to teach LLMs to prefer one answer over another, i.e., learning to have a preference.

PO is to fit the following kind of dataset

$$\{(x, y_w, y_l)\},$$

where x is the prompt, y_w is the preferred answer over y_l .

After PO step, we expect

$$\mathbb{P}_{\theta}(y_w|x) > \mathbb{P}_{\theta}(y_l|x).$$

That is, the finetuned model prefers y_w over y_l .

DPO: Direct Preference Optimization

- ▶ One very elegant way to performing PO is the **DPO**.
- ▶ DPO is based on BT model to measure preference.

DPO seeks to solve the following problem

$$\underset{\theta}{\text{minimize}} \left\{ -\log \sigma \left(\beta \log \left(\frac{\mathbb{P}_{\theta}(\mathbf{y}_w|\mathbf{x})}{\mathbb{P}_{ref}(\mathbf{y}_w|\mathbf{x})} \right) - \beta \log \left(\frac{\mathbb{P}_{\theta}(\mathbf{y}_l|\mathbf{x})}{\mathbb{P}_{ref}(\mathbf{y}_l|\mathbf{x})} \right) \right) \right\}.$$

- ▶ $\mathbb{P}_{ref}$ is the reference model after SFT, σ is the sigmoid (logistic) function.

Interpretation: DPO tries to **maximize the generation probability of $\mathbf{y}_w$** , while **simultaneously minimizing the generation probability of $\mathbf{y}_l$** .

- ▶ Though DPO is widely used and has strong performance, it does not do "unlearning" properly. $\rightsquigarrow$ We still have much to investigate in the future.

Some Future Research Directions

Here are some ongoing research directions we summarized:

- ▶ Memory efficient and efficient finetuning algorithms.
- ▶ Efficient and stable PO method, even DPO has many drawbacks.
- ▶ Speed up inference.
- ▶ Lightweight LLMs used on edge (like phone), using quantization, etc.
- ▶ Chain of thoughts.
- ▶ Important understanding of LLMs.
- ▶ ...

↪ Next lecture: More on LLMs, especially reasoning model.